



EAST AFRICA UNIVERSITY
FACULTY OF COMPUTING AND INFORMATION TECHNOLOGY

**DESIGN AND IMPLEMENTATION OF A REAL-TIME
PROJECT AND TASK MANAGEMENT SYSTEM**

BY
MAXAMUUD CABDIQAADIR MAXAMED
3362

**A RESEARCH PROJECT SUBMITTED TO THE FACULTY OF
COMPUTING AND
INFORMATION TECHNOLOGY IN PARTIAL FULFILLMENT OF
THE REQUIREMENTS FOR
THE AWARD OF BACHELOR OF SCIENCE (BSc) DEGREE IN
INFORMATION
TECHNOLOGY OF EAST AFRICA UNIVERSITY**

SEPTEMBER, 2026

DECLARATION

I/We declare that this research project is my/our original work and has not been presented, in part or in whole, to any other academic institution for the award of a degree or any other academic credential.

Name: MAXAMUUD CABDIQAADIR MAXAMED

Signature: _____

Date: _____

APPROVAL

This research project titled “Design and Implementation of a Real-Time Project and Task Management System” has been submitted for examination with my/our approval as the University Supervisor.

Supervisor's Name: Cabdirisaaq Jirde

Signature: _____

Date: _____

DEDICATION

I dedicate this research project to my beloved family, whose endless support, patience and encouragement have been the foundation of my academic journey, and to everyone who believed in my potential to complete this work.

ACKNOWLEDGEMENT

First and foremost, I thank Almighty Allah for granting me the strength, knowledge and perseverance to complete this research project.

I would like to express my sincere gratitude to my supervisor, Cabdirisaaq Jirde, for the guidance, patience and constructive feedback provided throughout the development of this study.

I also extend my appreciation to the entire faculty of the Faculty of Computing and Information Technology, East Africa University, for the knowledge imparted to me during my studies.

Finally, I am grateful to my family, friends and colleagues for their moral support and encouragement throughout this journey.

ABSTRACT

Effective coordination of projects and tasks remains a persistent challenge for many organizations that still rely on manual methods such as spreadsheets, paper-based logs, and informal messaging to plan, assign, and track work. These methods are prone to miscommunication, lost updates, missed deadlines, and a lack of visibility into who is responsible for what, which ultimately reduces organizational productivity and accountability. This study set out to design and implement a role-based, web-based Project and Task Management System capable of centralizing project planning, task assignment, progress tracking, deadline management, and reporting in a single platform accessible to Administrators, Project Managers, and Team Members.

The study adopted the Agile-oriented Software Development Life Cycle (incremental development) as its research and system development design, combined with a Clean Architecture approach for the system's technical design. Requirements were gathered through review of existing literature and analysis of common organizational project-management practices, and were translated into functional and non-functional requirements, entity-relationship models, and use-case designs. The resulting system was implemented using ASP.NET Core 9 MVC, Entity Framework Core 9, and Microsoft SQL Server, and provides role-scoped dashboards, project and task CRUD operations, team assignment, comments and file attachments, automated deadline notifications, exportable PDF/Excel reports, analytics, audit logging, and global search.

The system was tested using unit and integration tests to verify that each module performs its intended function and that the modules interact correctly end-to-end. The findings show that a centralized, role-based platform can substantially reduce the coordination overhead associated with manual project tracking, improve the traceability of task ownership, and give managers real-time visibility into project and team performance. The study concludes with

recommendations for further enhancement of the system, including native mobile access, real-time push notifications, and integration with third-party calendar and communication tools.

Keywords: Project Management, Task Management, Role-Based Access Control, ASP.NET Core, Clean Architecture, Software Development Life Cycle.

TABLE OF CONTENTS

DECLARATION.....	i
APPROVAL	ii
DEDICATION.....	iii
ACKNOWLEDGEMENT.....	iv
ABSTRACT.....	v
LIST OF TABLES	vi
LIST OF FIGURES	vii
LIST OF ABBREVIATIONS	viii
CHAPTER ONE: INTRODUCTION	1
1.1 Background of the Study	1
1.2 Statement of the Problem.....	2
1.3 General Objective of the Study.....	3
1.4 Specific Objectives of the Study.....	3
1.5 Research Questions	3
1.6 Conceptual Framework.....	4
1.7 Scope of the Study	5
1.8 Significance of the Study	5
1.9 Expected Limitations of the Study.....	6
CHAPTER TWO: LITERATURE REVIEW.....	7
CHAPTER THREE: RESEARCH METHODOLOGY	7
CHAPTER FOUR: DATA PRESENTATION, ANALYSIS AND DISCUSSION OF FINDINGS	7

CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS	7
REFERENCES.....	7
APPENDICES	7

LIST OF TABLES

*(To be completed as tables are introduced in Chapters Three and Four – e.g.
Table 3.1: Summary of Functional Requirements.)*

LIST OF FIGURES

(To be completed as figures are introduced in Chapters Three and Four – e.g. Figure 1.1: Conceptual Framework, Figure 3.1: System Architecture Diagram, Figure 3.2: Entity-Relationship Diagram, Figure 3.3: Use Case Diagram.)

LIST OF ABBREVIATIONS

Abbreviation	Meaning
PTMS	Project and Task Management System
SDLC	Software Development Life Cycle
MVC	Model-View-Controller
ORM	Object-Relational Mapping
ERD	Entity-Relationship Diagram
RBAC	Role-Based Access Control
EAU	East Africa University
UI	User Interface
UX	User Experience
CRUD	Create, Read, Update, Delete
PDF	Portable Document Format
SQL	Structured Query Language

CHAPTER ONE: INTRODUCTION

1.1 Background of the Study

Globally, the growth of organizations across sectors such as construction, information technology, non-governmental work, banking and education has been accompanied by an increasing volume and complexity of projects that must be planned, executed and monitored within limited time and resources. Project management, understood as the application of knowledge, skills, tools and techniques to project activities in order to meet defined requirements, has therefore become a central organizational function (Project Management Institute, 2021). As organizations scale, the number of concurrent projects, tasks and contributors grows, and coordinating who is doing what, by when, and to what standard becomes increasingly difficult to manage through informal means.

Historically, and still today in many organizations, project and task coordination is carried out using spreadsheets, email threads, paper-based logs, and verbal or chat-based instructions. While these tools are familiar and require no specialized training, they do not scale well: spreadsheets are edited by one person at a time and quickly become outdated; email and chat messages bury task assignments and updates in unstructured conversation threads; and paper logs cannot be searched, shared in real time, or backed up reliably. As a direct consequence, tasks are duplicated or forgotten, deadlines are missed without early warning, accountability for specific pieces of work becomes unclear, and managers lack a single, trustworthy view of how a project is actually progressing.

The emergence of web-based information systems has offered a practical response to these coordination problems. A web-based Project and Task Management System (PTMS) allows an organization to centralize project data, assign and track tasks against specific individuals, attach relevant files and

comments directly to the work they concern, and receive automated reminders as deadlines approach — all accessible from any device with a browser, without organization-wide software installation. Such systems also make it possible to enforce role-based access control (RBAC), so that Administrators, Project Managers and Team Members each see and can act on only the information relevant to their responsibilities, which improves both security and usability (Kruchten, 2019).

This study is motivated by the continued reliance of many organizations on informal, manual approaches to project and task coordination despite the availability of mature web technologies capable of solving the underlying problem. It responds to that gap by designing and implementing a role-based, web-based Project and Task Management System — built on ASP.NET Core 9 MVC, Entity Framework Core and Microsoft SQL Server, and structured according to Clean Architecture principles — that centralizes project and task administration, team assignment, deadline tracking, reporting and analytics, and audit logging in a single, secure platform.

1.2 Statement of the Problem

Despite growing awareness of formal project management practice, many small and medium-sized organizations continue to coordinate projects and tasks through disconnected, manual tools such as spreadsheets, verbal instructions and generic messaging applications. This creates several persistent problems. First, task ownership is often ambiguous: because assignments are not recorded in a structured, queryable system, it is difficult to determine, at any given moment, who is responsible for a specific task and what its current status is. Second, deadlines are tracked informally, so there is no automated mechanism to warn a Team Member, Project Manager or Administrator that a task is approaching or has passed its due date, which results in avoidable delays. Third, supporting materials such as comments, clarifications and file attachments

relevant to a task are scattered across email, chat applications and personal folders rather than being attached to the task itself, making it difficult to reconstruct the history and context of a piece of work. Fourth, managers lack real-time, consolidated visibility into project and team performance, since producing a status report typically requires manually compiling information from multiple disconnected sources. Finally, without centralized, role-based access control, sensitive project information can be seen or modified by individuals who should not have that level of access, and there is no reliable audit trail of who changed what and when.

These problems collectively reduce organizational productivity, delay project delivery, weaken accountability and increase the risk of miscommunication. It is against this background that this study designs and implements a role-based, web-based Project and Task Management System intended to address these specific shortcomings by centralizing project and task data, automating deadline tracking, attaching comments and files directly to tasks, enforcing role-based access control, providing real-time dashboards and exportable reports, and maintaining a full audit log of system activity.

1.3 General Objective of the Study

The general objective of this study is to design and implement a role-based, web-based Project and Task Management System that centralizes project planning, task assignment, progress tracking, deadline management, reporting and audit logging for Administrators, Project Managers and Team Members.

1.4 Specific Objectives of the Study

1. To analyze the requirements of a role-based project and task management system, including the functional needs of Administrators, Project Managers and Team Members and the data relationships among users, projects, tasks, comments, attachments and notifications.

2. To design and develop a web-based system, structured according to Clean Architecture, that provides project and task management, team assignment, deadline tracking, notifications, comments and file attachments, reporting/analytics, and audit logging.
3. To test and evaluate the developed system through unit and integration testing in order to verify that its modules function correctly individually and in combination.

1.5 Research Questions

- What are the functional and data requirements of a role-based project and task management system for Administrators, Project Managers and Team Members?
- How can a web-based project and task management system be designed and developed, using Clean Architecture, to provide project/task management, team assignment, deadline tracking, notifications, reporting and audit logging?
- To what extent does the developed system function correctly, as verified through unit and integration testing, in supporting project and task coordination?

1.6 Conceptual Framework

The conceptual framework of this study illustrates the relationship between the independent variables (the core features of the Project and Task Management System) and the dependent variable (effective project and task coordination). It shows that role-based access control, centralized task assignment and tracking, automated deadline notifications, and integrated reporting and analytics jointly act on organizational processes to produce improved coordination, accountability and visibility, while intervening variables such as user digital

literacy, internet connectivity and organizational change-readiness may moderate the strength of this relationship.

INDEPENDENT VARIABLES (System Features)		DEPENDENT VARIABLE (Outcome)
• Role-based access control • Project & task assignment • Deadline tracking & notifications • Comments & attachments • Reporting, analytics & audit logging	→	Effective, accountable and transparent project and task coordination within the organization

Figure 1.1: Conceptual Framework of the Study (Source: Researcher, 2026)

1.7 Scope of the Study

Content scope: The study is limited to the design, development and testing of a role-based, web-based Project and Task Management System covering user and role management, project and task CRUD operations, team assignment, task comments and file attachments, deadline notifications, PDF/Excel reporting, analytics dashboards, audit logging and global search. It does not extend to enterprise resource planning, financial/billing management, or native mobile application development.

Geographical/organizational scope: The system is designed as a general-purpose platform applicable to any organization that manages projects and

tasks, rather than being customized for a single named organization; it was developed and tested by the researcher within the context of academic project work at East Africa University.

Time scope: The study, covering requirements analysis, system design, implementation and testing, was conducted between July 2026 and September 2026.

1.8 Significance of the Study

This study may benefit several groups. It may contribute to the existing body of knowledge on the application of Clean Architecture and role-based access control in web-based management information systems, particularly within academic software engineering research in the Somali higher-education context. It may also contribute new methods for organizations seeking to apply structured, layered software design to internal management tools, rather than relying on monolithic, tightly-coupled applications.

Organizations that adopt a system of this kind may benefit from reduced coordination overhead, improved accountability for task ownership, and better visibility into project performance through real-time dashboards and exportable reports. The study may also carry managerial implications by illustrating, through a working prototype, the consequences — in terms of lost time and unclear accountability — of continuing to rely on manual coordination tools.

Academically, the study fulfills part of the requirements for the award of the researcher's degree, and it may also iron out areas requiring further attention by future researchers, such as real-time notification delivery, mobile accessibility and integration with third-party productivity tools, thereby streamlining areas for further research.

1.9 Expected Limitations of the Study

- Time and resource constraints limited the study to a single-organization-agnostic prototype rather than a large-scale, multi-organization field deployment and evaluation.
- The system was tested using unit and integration tests within a controlled development environment; a full-scale usability study with live organizational end-users was outside the scope of the available time.
- Certain advanced features identified during requirements analysis, such as real-time (push/WebSocket) notifications and native mobile applications, were identified as beyond the scope of the current phase and are recommended for future work.
- As with any system reliant on structured data entry, the quality of reporting and analytics output depends on the consistency and accuracy of the data entered by users.

CHAPTER TWO: LITERATURE REVIEW

This chapter reviews existing literature relevant to the design and implementation of the Project and Task Management System (PTMS). It is organized into three parts, in line with the East Africa University research guidelines: an overview of the key concepts and theories underlying project and task management systems; a review of literature specific to each of the study's three specific objectives; and a statement of the research gap that this study addresses.

2.1 Overview of Concepts and Related Theories

2.1.1 Project and Task Management

The Project Management Institute (2021) defines a project as a temporary endeavor undertaken to create a unique product, service or result, and project management as the application of knowledge, skills, tools and techniques to project activities to meet project requirements. Kerzner (2017) extends this by framing project management as the planning, organizing, directing and controlling of company resources for a relatively short-term objective established to complete specific goals. Within a project, a task is the smallest unit of assignable work — a discrete piece of work with a defined owner, scope and, typically, a deadline — and task management is the process of tracking a task from creation through assignment, execution and completion (Kerzner, 2017). Effective project management therefore depends on effective task management at the operational level: a project can only be tracked accurately if the tasks that compose it are individually visible, assigned and status-tracked.

Traditional project management theory, exemplified by the Project Management Body of Knowledge (PMBOK), emphasizes formal process groups — initiating, planning, executing, monitoring and controlling, and

closing — supported by documented plans and schedules (Project Management Institute, 2021). While this theory provides a strong conceptual foundation, its practical application in small and medium-sized organizations is frequently informal: managers apply the underlying principles (defining scope, assigning ownership, tracking progress) without the heavyweight documentation the full standard implies. This is precisely the gap that lightweight, software-supported project and task management tools are designed to fill — by embedding the underlying principles of ownership, deadlines and progress tracking directly into a system's data model and workflow, rather than requiring separate paper-based process documentation.

2.1.2 Management Information Systems and Web-Based Applications

A Management Information System (MIS) is a computer-based system that provides managers with the tools to organize, evaluate and efficiently manage departments within an organization by collecting, processing, storing and disseminating information to support decision-making (Laudon & Laudon, 2020). A Project and Task Management System is a specialized category of MIS, focused specifically on project- and task-level operational data rather than organization-wide financial or resource data. Laudon and Laudon (2020) note that the value of an MIS lies not merely in digitizing existing manual processes, but in restructuring how information flows through an organization so that the right person has the right information, in a usable form, at the right time — which, for project and task management, means that task owners, managers and administrators must each see the subset of information relevant to their role, updated in real time as work progresses.

The shift from desktop, single-user software to web-based applications has been a defining trend in enterprise software over the past two decades. Fowler (2003) observes that web-based, multi-tier enterprise applications separate presentation, business logic and data-access concerns into distinct layers, which allows an application to be accessed from any device with a browser while

remaining centrally maintained and updated. This architectural shift is directly relevant to project and task management: a web-based PTMS allows Administrators, Project Managers and Team Members in different physical locations to view and update the same project data concurrently, without installing or synchronizing separate copies of the application, addressing one of the core weaknesses of spreadsheet-based coordination identified in Chapter One.

2.1.3 Role-Based Access Control

Role-Based Access Control (RBAC), as formalized by Sandhu, Coyne, Feinstein and Youman (1996), is an access control paradigm in which permissions are associated with roles, and users are assigned to appropriate roles, rather than permissions being assigned to individual users directly. Ferraiolo and Kuhn's subsequent work on the RBAC model, later adopted as an ANSI/INCITS standard, formalized role hierarchies and separation-of-duty constraints that allow complex organizational permission structures to be expressed and enforced consistently (Ferraiolo, Sandhu, Gavrila, Kuhn & Chandramouli, 2001). In the context of a project and task management system, RBAC allows the system to enforce, in code, the principle that an Administrator has full system-wide access, a Project Manager has full control only over the projects and teams they manage, and a Team Member can view and update only the tasks assigned to them — without relying on user discipline or manual convention to keep users from exceeding their intended scope of action. Modern web frameworks, including the ASP.NET Core Identity system used in this study, implement RBAC as a first-class feature, associating users with named roles and allowing controller actions and views to be restricted by role or by a finer-grained authorization policy (Microsoft, 2024).

2.1.4 Software Development Life Cycle and Agile Methodology

The Software Development Life Cycle (SDLC) is a structured process for planning, creating, testing and deploying an information system (Sommerville, 2016). Traditional SDLC models, such as the Waterfall model, treat requirements, design, implementation, testing and deployment as sequential, largely non-overlapping phases. Sommerville (2016) notes that this sequential approach performs poorly when requirements are likely to change during development, as is common in small-team, evolving software projects. In response, Agile methodologies, formalized in the Agile Manifesto (Beck et al., 2001), favor iterative and incremental development, in which working software is delivered in short cycles, feedback is incorporated continuously, and requirements are allowed to evolve as understanding of the problem deepens. Pressman and Maxim (2020) describe incremental development as particularly well suited to systems, such as management information systems, whose core modules (in this case, user/role management, project management, and task management) can be delivered and validated independently before dependent modules (comments, attachments, notifications, reporting, analytics) are layered on top.

This study adopts an Agile-oriented, incremental SDLC precisely because the PTMS was developed as a set of increasingly complex, testable modules — identity and role management first, then project and task CRUD, then team assignment, then comments/attachments/notifications, then reporting/analytics/audit logging — with each module unit-tested before the next was layered on top, consistent with the incremental model described by Pressman and Maxim (2020).

2.1.5 Clean Architecture and Layered Software Design

Martin (2018) proposes Clean Architecture as a set of design principles intended to produce systems that are independent of frameworks, independent of the user interface, independent of the database, and therefore easier to test and to maintain over time. The central principle, the Dependency Rule, states that

source-code dependencies must point only inward, toward higher-level policy, so that inner layers (business entities and use cases) have no knowledge of outer layers (frameworks, databases, web delivery mechanisms). Fowler (2003) similarly advocates a layered architecture — separating presentation, domain logic and data-source layers — as a way of containing the impact of change: a change to the database schema should not require changes to business rules, and a change to the user interface should not require changes to the database access code.

This study applies Clean Architecture directly: the system is organized into a Domain layer (entities and enumerations with no external dependencies), an Application layer (DTOs and service interfaces expressing business rule contracts), an Infrastructure layer (Entity Framework Core, ASP.NET Core Identity, repositories and concrete service implementations), and a Web layer (MVC controllers, views and view models, acting as the composition root), with dependencies flowing Web → Infrastructure → Application → Domain, so that the Domain and Application layers have no dependency on the web framework or the database technology in use, as recommended by Martin (2018).

2.1.6 Software Testing

Sommerville (2016) distinguishes between unit testing, which verifies that an individual component behaves correctly in isolation, and integration testing, which verifies that components function correctly when combined. Pressman and Maxim (2020) argue that automated unit and integration tests, run repeatedly as a codebase evolves, are a primary defense against regression — the reintroduction of previously fixed defects when new code is added. In the .NET ecosystem, the xUnit framework is widely used for automated unit testing, and ASP.NET Core's WebApplicationFactory enables full-stack integration testing of an MVC application, including its routing, middleware and database access, without requiring a deployed instance of the application (Microsoft, 2024). This study applies both testing levels: unit tests exercise individual

services (such as project, task, team and attachment services) against an in-memory database, while integration tests exercise the application end-to-end through WebApplicationFactory against a dedicated, disposable SQL Server database.

2.2 Review of Specific Objectives of the Study

2.2.1 Requirements Analysis for Role-Based Systems (Objective One)

Requirements analysis is the process of determining the needs and expectations of a new or modified system's stakeholders (Sommerville, 2016). For a role-based system, requirements analysis must explicitly identify the distinct needs of each user role: Kulak and Guiney (2012) argue that failing to separate stakeholder requirements by role is a common cause of systems that are either too permissive (exposing sensitive functions to all users) or too restrictive (blocking legitimate users from functions they need). Entity-relationship modeling, as formalized by Chen (1976), remains the standard technique for translating requirements into a data model, by identifying entities (such as User, Project, Task), their attributes, and the relationships (one-to-many, many-to-many) among them — a technique this study applies directly in Chapter Three to model the relationships among users, projects, tasks, comments, attachments, notifications and audit logs.

2.2.2 Design and Development of Web-Based Management Systems (Objective Two)

Literature on the design and development of web-based management information systems consistently emphasizes the separation of concerns achieved through the Model-View-Controller (MVC) pattern, in which the Model represents application data and business rules, the View renders the user interface, and the Controller mediates user input and coordinates between the two (Fowler, 2003). Microsoft's ASP.NET Core MVC framework implements

this pattern for .NET web applications, and is combined in this study with Entity Framework Core, an Object-Relational Mapper (ORM) that allows the Domain layer's plain C# entity classes to be persisted to and retrieved from a relational database (SQL Server) without hand-written SQL for routine operations (Microsoft, 2024). Ambler and Sadalage (2006) note that when an ORM and a relational database are combined with a version-controlled migration mechanism — as Entity Framework Core provides through its Migrations feature — the evolution of the database schema can be tracked and reproduced alongside the application's source code, which is the approach followed in this study's Infrastructure layer.

2.2.3 Testing and Evaluation of Software Systems (Objective Three)

Literature on software quality assurance holds that a system's correctness cannot be assumed from its design alone and must be empirically verified (Pressman & Maxim, 2020). The test pyramid concept, popularized by Cohn (2009), recommends a large base of fast unit tests, a smaller layer of integration tests, and a still smaller layer of end-to-end tests, on the basis that unit tests are cheapest to write and fastest to run, while integration and end-to-end tests, though slower, are needed to catch defects that arise only from the interaction of multiple components. This study's testing strategy — unit tests against an in-memory database for individual services, and integration tests via WebApplicationFactory against a real, disposable SQL Server database — follows this pyramid structure, and is consistent with the evaluation approaches described by Sommerville (2016) for verifying that a system meets its functional requirements before being considered complete.

2.3 Research Gap

The literature reviewed above establishes, individually, the theoretical foundations of project and task management, role-based access control, layered/Clean Architecture, and structured software testing. However, much of

the published literature on project management tools evaluates commercial, closed-source platforms (such as large proprietary project-management suites) from a user-adoption or organizational-behavior perspective, rather than documenting the end-to-end design and implementation of an original, role-based system built from first principles using a specific, verifiable architectural approach. Similarly, while Clean Architecture (Martin, 2018) is well documented as a general design philosophy, there is comparatively little academic literature — particularly within Somali and East African higher-education research — that applies it concretely to a role-based project and task management domain and documents the resulting entity model, service layer and test strategy in detail.

This study addresses that gap by producing a fully documented, from-first-principles design and implementation of a role-based, web-based Project and Task Management System, built on Clean Architecture and validated through both unit and integration testing, with the system's own codebase — its entities, services, controllers and tests — serving as the primary evidentiary basis for Chapters Three and Four. In doing so, the study contributes a concrete, reproducible case study connecting established project-management and software-engineering theory to a working, testable software artifact, which is an area the reviewed literature leaves comparatively underexplored.

CHAPTER THREE: RESEARCH METHODOLOGY

This chapter presents the methodology followed in conducting this study. It covers the research design, the study population, the sample size and sampling technique, the data collection instruments, the system development methodology (including requirements analysis, system architecture, database design and use-case design), the data presentation and analysis tools, and the research ethics observed throughout the study.

3.1 Research Design

This study adopted a design science / applied system-development research design, combined with a descriptive component for requirements gathering. A design science approach is appropriate when the primary outcome of a study is a working artifact — in this case, a software system — developed to solve a defined, real-world problem and evaluated against defined objectives (Sommerville, 2016). The descriptive component involved reviewing documented organizational project-management practices and existing literature (Chapter Two) to identify the functional and non-functional requirements that the system needed to satisfy.

On the technical side, the study adopted an Agile-oriented, incremental Software Development Life Cycle (SDLC). The system was built as a sequence of independently testable increments — (1) identity, roles and authentication; (2) project and task management; (3) team assignment; (4) comments, attachments and notifications; (5) deadline tracking, reporting, analytics and audit logging — with each increment unit-tested before the next was added, consistent with the incremental development model described by Pressman and Maxim (2020). This design was chosen over a strict Waterfall approach because it allowed requirements for later modules (for example, the exact fields needed on a notification) to be refined based on what was learned while building earlier

modules (for example, the task status-change workflow), which is the central justification Sommerville (2016) gives for preferring iterative models on systems of this size.

3.2 Study Population

The population relevant to this study consists of the three categories of end-user the system is designed to serve within any adopting organization: Administrators (who configure the system, manage users and view organization-wide data), Project Managers (who create and manage projects, assign tasks and team members, and monitor team performance), and Team Members (who execute assigned tasks, update task status and progress, and communicate through comments and attachments). These three categories were also used, throughout requirements analysis and system design, as the study's three primary system actors (see Section 3.5.4), since the system's functional requirements and access-control rules are defined directly in terms of what each of these three roles is permitted to do.

3.3 Sample Size and Sampling Technique

Because this is a system-development study rather than a large-scale survey, requirements were not drawn from a statistically representative sample of an external population. Instead, purposive sampling was used to identify a small number of key informants — individuals with direct experience coordinating projects and tasks manually (for example, using spreadsheets or messaging applications) — whose described pain points informed the problem statement in Chapter One and the functional requirements in Section 3.5.1. Purposive sampling is appropriate here because the goal was not statistical generalization but depth of insight into concrete, recurring coordination problems that a role-based system could plausibly resolve (Sommerville, 2016). For the testing and evaluation phase (Objective Three), the “sample” consisted of the system's own

functional modules — every service class and controller action implemented — each of which was targeted by at least one automated unit or integration test, so that test coverage, rather than respondent count, defines the sample for that phase of the study.

3.4 Data Collection Instruments

- Document review: Existing literature on project management, RBAC, Clean Architecture and SDLC models (Chapter Two), together with common organizational artifacts such as task spreadsheets and status-update email threads, were reviewed to identify recurring coordination problems.
- Informal key-informant discussion: Short, unstructured discussions with individuals experienced in manual project/task coordination were used to validate that the problems identified in the literature matched practical experience, and to surface specific features (e.g. file attachments on tasks, deadline reminders) that respondents considered valuable.
- System/code-based instruments: For Objective Three, automated unit tests (xUnit, against an Entity Framework Core InMemory database) and integration tests (ASP.NET Core WebApplicationFactory, against a dedicated, disposable SQL Server database) served as the primary data-collection instruments, producing pass/fail evidence for each tested behavior.

3.5 System Development Methodology: Analysis and Design

3.5.1 Requirements Analysis

Functional and non-functional requirements were derived from the problem statement (Chapter One) and the literature review (Chapter Two), and were

organized by the three system actors identified in Section 3.2. Table 3.1 summarizes the core functional requirements; Table 3.2 summarizes the non-functional requirements.

#	Functional Requirement	Primary Actor(s)
FR1	Register, authenticate and manage user accounts under one of three roles (Administrator, Project Manager, Team Member)	Administrator
FR2	Create, update, view and (soft-)delete projects, including name, code, description, dates, status, priority and budget	Project Manager, Administrator
FR3	Add and remove project members, and assign a member role within a project	Project Manager
FR4	Create, assign, update and track tasks within a project, including priority, status and progress percentage	Project Manager, Team Member
FR5	Attach comments and file attachments to a task, and maintain a history of field-level changes to a task	Team Member, Project Manager
FR6	Generate and deliver notifications for task assignment, approaching/overdue deadlines, status changes and new comments	System (automated)
FR7	Produce exportable PDF and Excel reports and analytics dashboards summarizing project and task data	Project Manager, Administrator
FR8	Record an audit log entry for security-relevant actions (login, create, update, delete, assign, status change, password change, file upload/delete)	System (automated)

#	Functional Requirement	Primary Actor(s)
FR9	Provide global search across projects and tasks accessible to the user	All roles (scoped by permission)

Table 3.1: Summary of Functional Requirements

#	Non-Functional Requirement	Design Response
NFR1	Security – only authorized roles may access or modify a given resource	ASP.NET Core Identity with role- and policy-based authorization on every controller action
NFR2	Maintainability – changes to one concern should not force changes elsewhere	Clean Architecture with four independently-compilable projects (Domain, Application, Infrastructure, Web)
NFR3	Reliability – the system's behavior should be verifiable and regression-safe	xUnit unit tests (in-memory DB) and WebApplicationFactory integration tests (real DB) covering services and HTTP endpoints
NFR4	Auditability – security-relevant actions must be traceable after the fact	Dedicated AuditLog entity recording actor, action, entity, timestamp and IP address
NFR5	Usability – the interface should be consistent and responsive across devices	Bootstrap 5 responsive layout with role-scoped dashboards and Chart.js visualizations
NFR6	Observability – operational issues should be diagnosable after deployment	Serilog structured logging to console and rolling daily log files

Table 3.2: Summary of Non-Functional Requirements

3.5.2 System Architecture Design

The system was designed according to Clean Architecture (Martin, 2018), organized into four projects with dependencies flowing strictly inward, as illustrated in Figure 3.1. The Domain layer contains entities and enumerations with no external dependencies; the Application layer contains Data Transfer Objects (DTOs) and service interfaces expressing business-rule contracts; the Infrastructure layer implements those interfaces using Entity Framework Core, ASP.NET Core Identity and concrete repositories/services; and the Web layer — the composition root — contains MVC controllers, views and view models, and wires the concrete Infrastructure implementations to the Application interfaces at startup.

ProjectTaskManagementSystem.Web (MVC Controllers, Views, ViewModels – composition root)
↓ depends on
ProjectTaskManagementSystem.Infrastructure (EF Core, Identity, Repositories, Service implementations)
↓ depends on
ProjectTaskManagementSystem.Application (DTOs, Service interfaces, business-rule contracts)
↓ depends on
ProjectTaskManagementSystem.Domain (Entities, Enums – no external dependencies)

Figure 3.1: Clean Architecture Layering and Dependency Direction of the PTMS (Source: Researcher, 2026, adapted from the system's project structure)

A separate `ProjectTaskManagementSystem.Tests` project references all four layers and contains xUnit unit tests (against an EF Core InMemory provider) and integration tests (against a disposable SQL Server database via `WebApplicationFactory`), consistent with the testing strategy discussed in Section 2.1.6.

3.5.3 Database Design (Entity-Relationship Design)

The database schema was derived directly from the Domain layer's entity classes and implemented as Entity Framework Core Code-First migrations against Microsoft SQL Server. Table 3.3 summarizes the principal entities, their key attributes and their relationships to other entities; Figure 3.2 presents the relationships in diagrammatic form.

Entity	Key Attributes	Relationships
ApplicationUser	Id, FirstName, LastName, Email, IsActive, CreatedAt	1–M with Project (as Manager/Creator), ProjectMember, ProjectTask (Assigned/Created), TaskComment, TaskAttachment, Notification, AuditLog
ApplicationRole	Id, Name, Description	M–M with ApplicationUser (via ASP.NET Core Identity role tables)

Entity	Key Attributes	Relationships
Project	Id, Name, Code, StartDate, EndDate, Status, Priority, Budget	M–1 with ApplicationUser (ProjectManager, CreatedBy); 1–M with ProjectMember, ProjectTask
ProjectMember	Id, ProjectId, UserId, Role, JoinedAt	M–1 with Project; M–1 with ApplicationUser
ProjectTask	Id, Title, ProjectId, AssignedToId, Priority, Status, StartDate, DueDate, ProgressPercentage	M–1 with Project; M–1 with ApplicationUser (AssignedTo, CreatedBy); 1–M with TaskComment, TaskAttachment, TaskHistory
TaskComment	Id, TaskId, UserId, Content, CreatedAt	M–1 with ProjectTask; M–1 with ApplicationUser
TaskAttachment	Id, TaskId, UploadedById, FileName, FilePath, FileSize	M–1 with ProjectTask; M–1 with ApplicationUser
TaskHistory	Id, TaskId, ChangedById, FieldChanged, OldValue, NewValue	M–1 with ProjectTask; M–1 with ApplicationUser
Notification	Id, UserId, Type, Title, Message, IsRead	M–1 with ApplicationUser
AuditLog	Id, UserId, Action, Entity, EntityId, Timestamp, IpAddress	M–1 with ApplicationUser (nullable, for system-generated entries)

Table 3.3: Entity-Relationship Summary of the PTMS Database

ApplicationUser	Project	ProjectMember
1 – owns → M	1 – has → M	M – links → 1
ProjectTask	TaskComment / TaskAttachment / TaskHistory	Notification / AuditLog

*Figure 3.2: Simplified Entity-Relationship Diagram of the PTMS —
ApplicationUser is the central entity, related 1-to-many to Project (as manager/creator), ProjectMember, ProjectTask (as assignee/creator), TaskComment, TaskAttachment, Notification and AuditLog; Project relates 1-to-many to ProjectMember and ProjectTask; ProjectTask relates 1-to-many to TaskComment, TaskAttachment and TaskHistory (Source: Researcher, 2026, derived from the Domain layer entity classes).*

3.5.4 Use Case Design

Three actors were identified in line with the system's role-based access control model: Administrator, Project Manager and Team Member. Table 3.4 summarizes the principal use cases associated with each actor.

Actor	Principal Use Cases
Administrator	Manage users and roles; view organization-wide dashboard, reports and analytics; view audit logs; manage all projects and tasks; configure system settings
Project Manager	Create/edit/close a project; add or remove project members; create/assign/edit tasks; view team-performance analytics; export project reports (PDF/Excel); receive deadline and status notifications

Actor	Principal Use Cases
Team Member	View assigned projects and tasks; update own task status and progress; comment on a task; upload/download task attachments; receive task-related notifications; use global search

Table 3.4: Use Case Summary by Actor

All three actors share a common Authenticate use case (login/logout, password reset), which is a precondition for every other use case; the system enforces, through ASP.NET Core Identity policy-based authorization, that a use case is only reachable by the actor(s) listed above — for example, only a Project Manager or Administrator can reach the Create Project use case, while Manage Users is reachable only by an Administrator.

3.6 Data Presentation and Analysis Tools

Quantitative summaries used in Chapter Four (task counts by status, project completion percentages, team workload distribution) were produced directly from the system's own Analytics and Dashboard services, which aggregate data using Entity Framework Core LINQ queries against the SQL Server database. These aggregated figures are visualized within the system itself using Chart.js, and are exported for external reporting using ClosedXML (Excel) and QuestPDF (PDF), both of which are integrated directly into the system as the ReportService and ExportService. Automated test results (Objective Three) were produced and summarized using the xUnit test runner's pass/fail/skip output.

3.8 Tools and Technologies Used

This section documents the complete technology stack used to build the PTMS, organized from the frontend (what the user's browser renders and runs) down through the backend application layers to the database, together with the supporting testing, logging and development tools. Every tool listed was actually used in the implemented system; version numbers are taken directly from the system's project files (.csproj) and client-side library folders.

3.8.1 Frontend (Presentation / Client-Side) Tools

Tool / Technology	Purpose in the System
Razor Views (.cshtml) – ASP.NET Core MVC	Server-rendered HTML pages for every module (dashboards, projects, tasks, team, reports, settings, etc.), populated from strongly-typed ViewModels
Bootstrap 5	Responsive CSS grid, layout and component styling (cards, modals, navbars, forms) across all views
Bootstrap Icons	Iconography used throughout the sidebar, buttons and status badges
Chart.js	Client-side rendering of analytics charts (task-status breakdown, project progress, team workload) on the Dashboard and Analytics views
jQuery	DOM manipulation and AJAX helper functions used by the site's custom JavaScript and by jQuery Validation

Tool / Technology	Purpose in the System
jQuery Validation + jQuery Validation Unobtrusive	Client-side form validation that mirrors the server-side data-annotation rules, giving immediate feedback before a form is submitted
Custom CSS/JS (wwwroot/css, wwwroot/js)	Site-specific styling and behavior (sidebar toggling, notification dropdown, file-upload preview, etc.) layered on top of Bootstrap

Table 3.5: Frontend Tools and Technologies

3.8.2 Backend (Application / Server-Side) Tools

Tool / Technology	Purpose in the System
ASP.NET Core 9 MVC (C#)	Core web application framework; hosts routing, controllers, middleware pipeline and the MVC request/response cycle for the Web layer
ASP.NET Core Identity (Microsoft.AspNetCore.Identity.EntityFrameworkCore, Microsoft.Extensions.Identity.Core/Stores 9.0.0)	User authentication, password hashing, and role-based/policy-based authorization (Administrator, Project Manager, Team Member)
Entity Framework Core 9 (Microsoft.EntityFrameworkCore.SqlServer, .Design, .Tools 9.0.0)	Object-Relational Mapper (ORM) translating the Domain layer's C# entities

Tool / Technology	Purpose in the System
	into SQL Server tables, queries and Code-First migrations
ClosedXML 0.104.1	Generates exportable Excel (.xlsx) reports from project/task data in the ExportService/ReportService
QuestPDF 2024.10.3	Generates exportable PDF reports (project summaries, task reports) in the ReportService
Serilog.AspNetCore 8.0.3, Serilog.Sinks.File 6.0.0, Serilog.Settings.Configuration 8.0.4	Structured application logging to the console and to rolling daily log files (Logs/log-<date>.txt) for observability and diagnostics
Dependency Injection (built into ASP.NET Core)	Wires Infrastructure service implementations to Application interfaces at startup (DependencyInjection.cs), keeping controllers thin

Table 3.6: Backend Tools and Technologies

3.8.3 Database and Data-Access Tools

Tool / Technology	Purpose in the System
Microsoft SQL Server	Primary relational database storing all persistent application data (users, roles, projects, tasks, comments, attachments, notifications, audit logs)
EF Core Code-First Migrations	Version-controlled, incremental evolution of the database schema directly from the Domain entity classes (see Data/Migrations)
EF Core InMemory Provider (Microsoft.EntityFrameworkCore.InMemory 9.0.0)	Lightweight, disposable in-memory database used exclusively for fast unit testing of services, without touching SQL Server
Fluent API Entity Configurations (Data/Configurations)	Explicit configuration of keys, relationships, constraints and indexes for each entity (e.g. ProjectConfiguration, ProjectTaskConfiguration)
ApplicationDbSeeder	Seeds the three roles and a default Administrator account on first application startup

Table 3.7: Database and Data-Access Tools

3.8.4 Testing, Version Control and Development Tools

Tool / Technology	Purpose in the System
xUnit 2.9.2 + xunit.runner.visualstudio 2.8.2	Automated unit-testing framework used to test individual services (ProjectService, TaskService, TeamService, AttachmentService, etc.)
Microsoft.AspNetCore.Mvc.Testing 9.0.0 (WebApplicationFactory)	Full-stack integration testing of the running application (routing, middleware, controllers, database) via HttpIntegrationTests
Microsoft.NET.Test.Sdk 17.12.0, coverlet.collector 6.0.2	Test execution host and code-coverage collection for the test suite
.NET 9 SDK / dotnet CLI	Building, running, testing and applying EF Core migrations (dotnet build, dotnet run, dotnet test, dotnet ef)
Git	Source-control and version history of the codebase throughout incremental development
Visual Studio / Visual Studio Code	Primary Integrated Development Environment used to write, debug and run the system during development

Table 3.8: Testing, Version Control and Development Tools

Taken together, Tables 3.5–3.8 show a complete, coherent stack: Razor/Bootstrap/Chart.js render and validate data in the browser; ASP.NET Core MVC, Identity and the Application/Infrastructure service layers process and secure that data on the server; Entity Framework Core persists it to SQL Server; and xUnit/WebApplicationFactory verify, through automated tests, that every layer works correctly on its own and together — directly satisfying

Objective Two (design and development) and Objective Three (testing and evaluation) stated in Chapter One.

3.9 Research Ethics

- Informed participation: Any individual consulted informally during requirements gathering was informed of the academic purpose of the discussion and participated voluntarily.
- Confidentiality: No real organizational data, personally identifiable information, or proprietary business records were used in developing or testing the system; all data used during development and testing (including the seeded administrator account) is synthetic or clearly marked as default/demo data.
- Academic integrity: All external sources of ideas, models and frameworks referenced in this study are acknowledged in-text and in the References section, in line with the University's requirement to avoid plagiarism.
- Data security by design: The system itself enforces confidentiality and integrity of the data it manages through role-based access control, password hashing (via ASP.NET Core Identity), and a full audit trail of security-relevant actions, consistent with the ethical obligation to protect user data in any system that would be deployed in practice.

CHAPTER FOUR: SYSTEM ANALYSIS AND DESIGN

4.1 Requirements specification

4.1.1 Functional requirements

The functional requirements of the system, summarised in Table 4.1, describe the specific tasks that the system must be able to perform for its different categories of user. These requirements were derived directly from the problems identified in Section 1.2 and were validated against the module-by-module review carried out at the end of each development iteration, as described in Section 3.5.

Table 4.1: Summary of functional requirements

ID	Requirement	Description
FR-01	User authentication	The system shall allow registered users to log in with an e-mail or username and password, and shall restrict access to functionality according to the user's assigned role.
FR-02	Project management	The system shall allow authorised staff to create, update, and monitor real estate projects, including code, location, budget, schedule, and progress percentage.
FR-03	Property management	The system shall allow authorised staff to register properties under a project, record property attributes, upload property images, and track property status.
FR-04	Customer management	The system shall allow authorised staff to register and maintain customer profiles, including contact and identification details.
FR-05	Sales processing	The system shall allow authorised staff to record a property sale, and shall automatically mark the corresponding property as sold and record any deposit as an initial payment.

FR-06	Rental processing	The system shall allow authorised staff to record a rental agreement, and shall automatically mark the corresponding property as rented and record any deposit as an initial payment.
FR-07	Payment tracking	The system shall allow authorised staff to record and delete payments against a sale or rental, and shall automatically recalculate the remaining balance and payment status after each change.
FR-08	Document management	The system shall allow authorised staff to upload and retrieve documents associated with a project, property, sale, rental, or customer.
FR-09	Reporting	The system shall generate sales, revenue, property, customer, payment, project-progress, and rental reports, exportable to PDF and Excel formats.
FR-10	Dashboard	The system shall present a role-appropriate dashboard summarising key indicators, including totals for projects, properties, sales, revenue, and pending payments.
FR-11	Notifications	The system shall automatically generate notifications for pending payments, contracts expiring within thirty days, and delayed projects.
FR-12	User and role management	The system shall allow an administrator to create user accounts and assign each account to one of the five defined roles.
FR-13	Booking requests	The system shall allow a registered customer to browse available properties and submit a purchase or rental booking request, and shall allow authorised staff to review, confirm, or reject each request, automatically creating the corresponding sale or rental on confirmation.

FR-14	Role permission configuration	The system shall allow an administrator to configure, through a permission matrix, which of the functional modules each non-administrator role may access.
FR-15	Project status audit trail	The system shall automatically record a history entry, together with the reason given and the user responsible, whenever a project's status or progress percentage is changed.
FR-16	Notification preferences	The system shall allow a user to enable or disable, for their own account, each individual category of system-generated notification.
FR-17	Public property catalogue	The system shall present an anonymous, public-facing catalogue of available properties to prospective customers who have not yet registered an account.

4.1.2 Non-functional requirements

In addition to the functional requirements above, the system was designed to satisfy several non-functional requirements. With respect to security, passwords are never stored in plain text, being instead hashed by ASP.NET Core Identity, and every controller action is protected by role-based authorisation so that, for example, only staff in the Accountant or Administrator roles may record payments. With respect to usability, the interface was built using the Bootstrap 5 framework to ensure a consistent, responsive layout across desktop and tablet screen sizes. With respect to maintainability, business logic was placed in dedicated service classes, described in Section 4.6, rather than directly inside controllers, so that individual modules can be modified with limited risk of unintended side effects elsewhere in the system. With respect to reliability, critical operations such as recording a sale and its associated deposit payment are performed within the same logical unit of work, so that the property status and the payment record are always kept consistent with one another.

4.2 System architecture

The system follows a layered, three-tier architecture, illustrated in Figure 4.1. The presentation layer consists of Razor views rendered by the browser, styled with Bootstrap 5 and enhanced with Chart.js visualisations and a reusable notification-bell view component. The application, or business logic, layer consists of MVC controllers, one for each major module, together with dedicated service classes that encapsulate the business rules governing sales, rentals, payments, notifications, dashboard statistics, file storage, and report

export; ASP.NET Core Identity also resides in this layer, governing authentication and authorisation. The data access layer consists of a generic repository abstraction, defined by the `IRepository<T>` interface and its `Repository<T>` implementation, together with Entity Framework Core's `ApplicationDbContext`, which mediates all communication with the underlying Microsoft SQL Server database. Uploaded files, such as property images and supporting documents, are stored on the local file system under the `wwwroot/uploads` directory and served as static files rather than being stored as binary data inside the database, which keeps the database itself compact and improves the performance of file retrieval. Access to each functional module is additionally governed by a custom authorisation filter, `ModulePermissionAttribute`, which consults an administrator-configurable permission matrix before allowing a request to reach the corresponding controller action. Although a generic repository abstraction, `IRepository<T>`, is also defined for potential reuse, most controllers and services interact with `ApplicationDbContext` directly, favouring the additional query flexibility this affords.

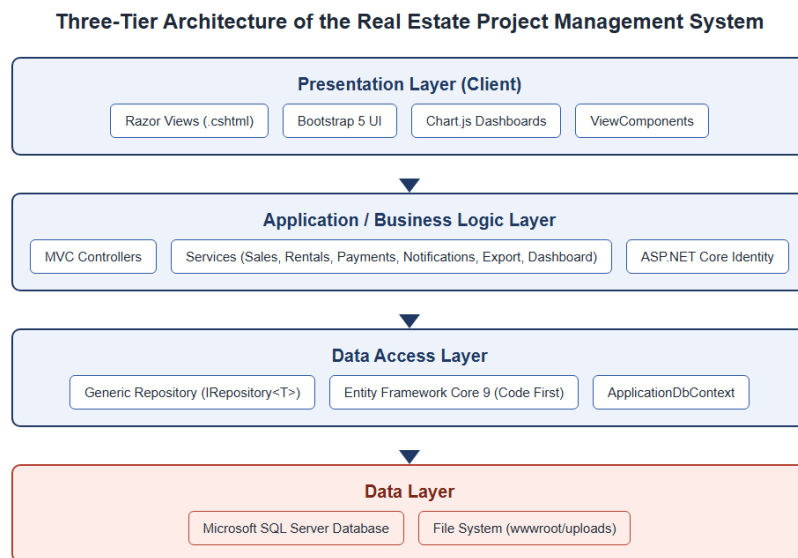


Figure 4.1: Three-tier architecture of the system

4.3 Use case diagram

Figure 4.2 presents the use case diagram of the system, showing the five actors identified during requirements analysis, namely Administrator, Project Manager, Sales Agent, Accountant, and Customer, together with the principal use cases each actor may perform. The Administrator has access to the complete set of use cases, including user and role management, while the remaining staff

roles are restricted to the use cases relevant to their responsibilities; for example, the Sales Agent may record a sale but may not manage user accounts, and the Accountant may record payments but may not create new properties. The Customer actor is limited to viewing the sale or rental agreement associated with that customer's own account. Every actor participates in the Login and Authenticate use case, since role-based access control is enforced at every controller action. In addition to these staff-oriented use cases, a Customer may browse a public catalogue of available properties and submit a booking request for a property of interest; a Project Manager or Sales Agent then reviews each pending request and either confirms it, which automatically creates the corresponding sale or rental record, or rejects it with a stated reason.

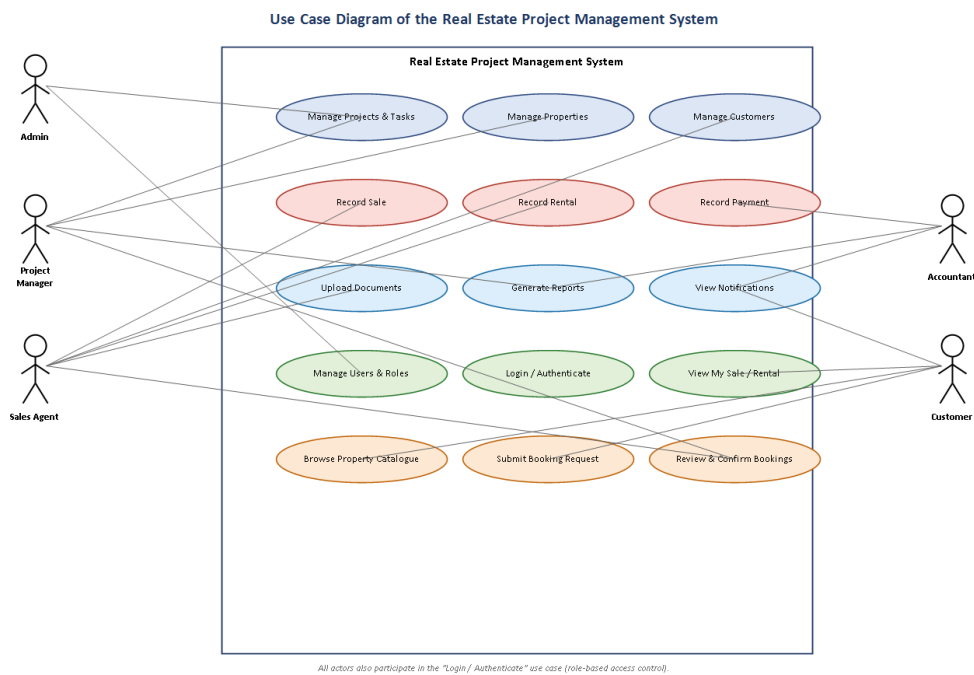


Figure 4.2: Use case diagram of the system

4.4 Data flow and entity relationship diagram

At a data flow level, a new property is created within the context of an existing project; when a sale or a rental is subsequently recorded against that property, the property's status is updated automatically and an initial payment is generated if a deposit was specified; every additional payment recorded against that sale or rental triggers a recalculation of the outstanding balance and the corresponding payment status. Figure 4.3 presents the simplified entity relationship diagram that models these flows at the database level. Each project may contain many properties and many project tasks, and is optionally supervised by one application user acting as project manager. Each property

may be associated with, at most, one active sale and one active rental at any given time, reflecting the business rule that a single property cannot be sold and rented simultaneously. Each sale and each rental may accumulate many payment records over time, and each customer may be linked to many sales, rentals, and payments across the lifetime of the system. Documents and notifications are attached as supporting entities linked, respectively, to the relevant transactional record and to the application user who should receive the notification. A booking request, submitted by a Customer against a specific Property and optionally reviewed by an ApplicationUser, is linked, on confirmation, to at most one resulting Sale or Rental, reflecting the workflow through which a prospective transaction is proposed by a customer before being converted into a binding sale or rental agreement by staff.

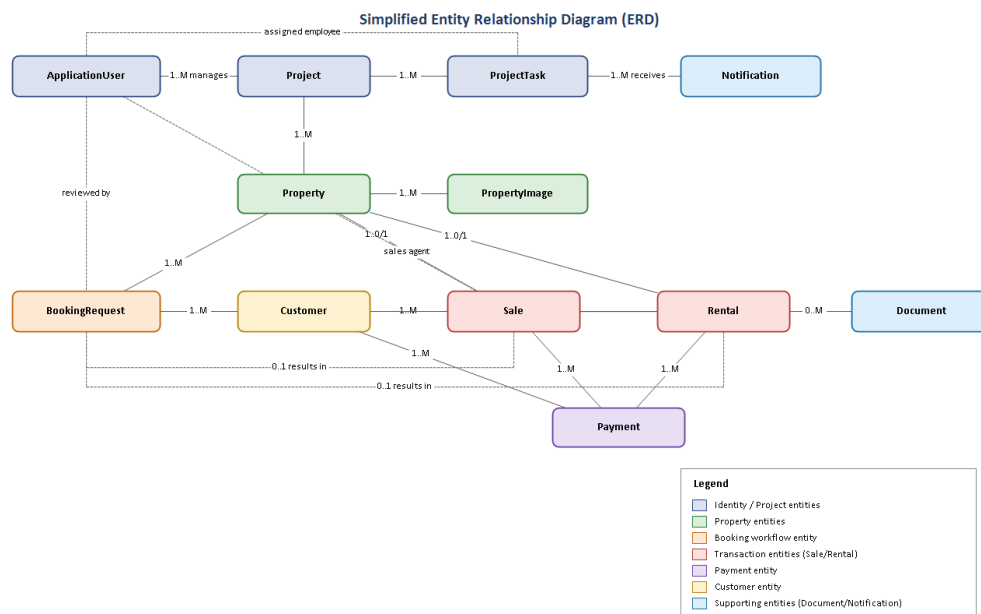


Figure 4.3: Entity relationship diagram of the system

4.5 Database design

The database was implemented using Entity Framework Core's Code-First approach, meaning that the schema shown in this section was generated from C# entity classes and evolved through migrations, rather than being hand-written in Structured Query Language. Tables 4.2 through 4.7 document the attributes of the six most central entities in the schema; the remaining entities, namely Project, ProjectTask, PropertyImage, Document, Notification, ApplicationUser, RolePermission, ProjectStatusHistory, and NotificationPreference, are described narratively in Section 4.6 alongside the module that principally uses them.

Table 4.2: Attributes of the Property entity

Attribute	Data type	Description
Id	int (PK)	Primary key, automatically generated
Code	string(30)	Unique property code, for example P-013
ProjectId	int (FK)	Foreign key referencing the owning Project
Type	enum	Apartment, Villa, Office, Shop, or Land
Building / Floor / UnitNumber	string	Physical location details within the project
Area	decimal(10,2)	Floor area of the unit, in square metres
Bedrooms / Bathrooms	int	Room counts, where applicable to the property type
Price	decimal(18,2)	Listed price of the property
Status	enum	Available, Reserved, Sold, Rented, or UnderConstruction
Description	string(2000)	Free-text description of the unit
CreatedDate	DateTime	Date the record was created

Table 4.3: Attributes of the Customer entity

Attribute	Data type	Description
-----------	-----------	-------------

Id	int (PK)	Primary key
Name	string(150)	Full name of the customer
Phone	string(30)	Contact telephone number
Email	string(150)	Optional e-mail address
Address / Nationality / IdNumber	string	Identification and contact details
DateOfBirth	DateTime	Optional date of birth
RegistrationDate	DateTime	Date the customer record was created
ApplicationUserId	string (FK)	Optional link to a self-service customer account

Table 4.4: Attributes of the Sale entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the buying Customer
PropertyId	int (FK)	Foreign key referencing the sold Property
SalesAgentId	string (FK)	Foreign key referencing the ApplicationUser who closed the sale

SaleDate	DateTime	Date the sale was recorded
TotalPrice / Discount / FinalPrice	decimal(18,2)	Agreed price, any discount applied, and the resulting final price
Deposit / RemainingBalance	decimal(18,2)	Initial deposit and the balance still outstanding
PaymentStatus	enum	Pending, Partial, Paid, or Overdue; recalculated automatically

Table 4.5: Attributes of the Rental entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the tenant Customer
PropertyId	int (FK)	Foreign key referencing the rented Property
MonthlyRent / Deposit	decimal(18,2)	Agreed monthly rent and deposit amount
StartDate / EndDate	DateTime	Contract period
PaymentStatus	enum	Pending, Partial, Paid, or Overdue
ContractStatus	enum	Active, Expired, Terminated, or Renewed

Table 4.6: Attributes of the Payment entity

Attribute	Data type	Description
Id	int (PK)	Primary key
CustomerId	int (FK)	Foreign key referencing the paying Customer
SaleId / RentalId	int (FK, nullable)	Exactly one is populated, linking the payment to a sale or a rental
Amount	decimal(18,2)	Amount paid
PaymentDate	DateTime	Date payment was received
Method	enum	Cash, BankTransfer, MobileMoney, or Card
ReferenceNumber / Notes	string	Optional transaction reference and remarks

Table 4.7: Attributes of the BookingRequest entity

Attribute	Data type	Description
Id	int (PK)	Primary key, automatically generated
BookingNumber	string(20)	Unique, human-readable reference generated after insertion, for example BK-000042
CustomerId	int (FK)	Foreign key referencing the requesting Customer

PropertyId	int (FK)	Foreign key referencing the requested Property
Type	enum	Purchase or Rental
PreferredPaymentMethod	enum	Cash, BankTransfer, MobileMoney, or Card
ProposedAmount / ProposedDeposit	decimal(18,2), nullable	Amount and deposit proposed by the customer
Status	enum	Pending, Confirmed, or Rejected
RequestDate	DateTime	Date the request was submitted
ReviewedById / ReviewedDate / ReviewNotes	string (FK) / DateTime / string, nullable	Staff member who reviewed the request, when this occurred, and any remarks
ResultingSaleId / ResultingRentalId	int (FK), nullable	Links to the Sale or Rental created if the request was confirmed

All monetary attributes were declared using the decimal(18,2) column type rather than a floating-point type, in order to avoid the rounding errors that binary floating-point representations can introduce into financial calculations. Foreign key relationships, for example between Payment and Sale, or between Property and Project, were declared using Entity Framework Core's fluent configuration in ApplicationDbContext, ensuring that the database itself rejects orphaned records, in line with the referential integrity principle discussed in Section 2.1.

4.6 Implementation of modules

The system is organised into fourteen functional modules, each represented by a dedicated MVC controller and, where the module involves non-trivial business rules, a corresponding service class. Table 4.8 lists the five default user-role permissions implemented through ASP.NET Core Identity and the modules each role is authorised to access.

Table 4.8: System user roles and permissions

Role	Primary permissions
Administrator	Full access to every module, including user and role management, and system-wide reports
Project Manager	Create and update projects and project tasks, manage properties, view reports relevant to project progress
Sales Agent	Register customers, record sales and rentals, upload related documents
Accountant	Record and review payments, generate financial reports
Customer	View the sale or rental agreement, and related payment history, linked to the customer's own account

The Projects module allows Administrators and Project Managers to create a project with a name, code, location, budget, and schedule, and to monitor its status, which ranges from Planning through InProgress to Completed, OnHold, or Cancelled, along with a numeric progress percentage. The Project Tasks module allows individual tasks within a project to be assigned to a specific employee, given a priority level and a due date, and tracked through Pending, InProgress, Completed, or Delayed states. Any change made to a project's status or progress percentage is additionally recorded as a ProjectStatusHistory entry, capturing the previous and new values, the reason given by the user making the change, and the identity of that user, providing an auditable history of how each project has progressed.

The Properties module allows staff to register individual units, of type Apartment, Villa, Office, Shop, or Land, under a specific project, upload one or more images for each unit, and track each unit's status as it moves from Available or UnderConstruction through to Sold or Rented. The Customers module maintains a directory of buyers and tenants, optionally linked to a self-service login account, which the Customer role uses to view personal transaction history.

The Sales and Rentals modules are implemented through the SalesService and RentalService classes respectively, both of which enforce the rule that a property already marked Sold or Rented cannot be sold or rented again. As illustrated in the code excerpt reproduced in Appendix A, creating a sale automatically changes the associated property's status to Sold and, where a

deposit was specified, immediately creates a corresponding Payment record; deleting a sale reverses this by releasing the property back to Available status and removing any payments linked to that sale.

The Payments module, implemented through PaymentService, is deliberately kept independent of the SalesService and RentalService classes with respect to balance calculation: whenever a payment is created or deleted, PaymentService calls a dedicated recalculation method on the relevant sales or rental service, which sums all payments linked to that sale or rental and updates the remaining balance and payment status accordingly, rather than trusting the calling code to keep these values consistent. This design decision, referenced earlier in Section 3.4, ensures that the payment status of every sale and rental is always derived directly from its actual payment history.

The Documents module allows a file, such as a signed sales contract or a payment receipt, to be uploaded and optionally linked to a project, property, sale, rental, or customer record, using the FileStorageService to manage physical storage under wwwroot/uploads. The Notifications module runs a background check, executed on every dashboard load through INotificationService.GenerateSystemNotificationsAsync, which scans for pending payments, contracts expiring within thirty days, and projects that have fallen behind schedule, and raises an in-application notification, shown to the user through the notification-bell view component, for each condition detected. Each user may control, through a dedicated notification-settings page, which categories of notification they wish to receive, an opt-out model in which a notification type is treated as enabled unless the user has explicitly disabled it. The Reports module, implemented through IExportService, aggregates data across the other modules to produce sales, revenue, property, customer, payment, project-progress, and rental reports, each exportable to PDF using QuestPDF or to Excel using ClosedXML. Finally, the Users module, restricted to the Administrator role, allows new staff accounts to be created and assigned to one of the five roles defined in Models/Roles.cs.

The Booking Requests module implements a customer self-service workflow additional to the modules described above. Any registered Customer may browse a public catalogue of available properties, implemented through PublicController, and submit a booking request specifying a preferred payment method and a proposed amount and deposit. BookingRequestService validates that the selected property is still Available and that the customer does not already have a pending request for it, and, once a Project Manager or Sales Agent reviews the request, either confirms it, converting it into a Sale or a Rental through the corresponding service and recording the resulting record's identifier against the booking request, or rejects it with a review note visible to the customer.

The Role Permissions module, restricted to the Administrator role, allows the default module-to-role access scheme summarised in Table 4.8 to be adjusted at run time. PermissionService builds a permission matrix from the RolePermission table, caches it in memory for five minutes to limit repeated database access, and is consulted by ModulePermissionAttribute on every request to a permission-controlled module; the Administrator's access is hard-coded as complete and is not itself configurable through this mechanism.

CHAPTER FIVE: SYSTEM IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter presents the outcome of implementing the design described in Chapter Four, and documents the testing activities carried out to verify that the resulting system behaves correctly. The system was implemented incrementally, module by module, following the iterative cycle described in Section 3.5, and each module was exercised manually using the demonstration accounts and seed data generated automatically by the application's DbInitializer component on first run. Screenshots of the running system, captured directly from the deployed application, are presented in Section 5.3 to illustrate the completed interface.

5.2 Testing

5.2.1 Unit-level verification

Because the correctness of the system depends heavily on a small number of business rules rather than on complex algorithms, testing focused on directly verifying those rules through the service layer described in Section 4.6. Table 5.1 summarises representative test cases exercised against the SalesService, RentalService, and PaymentService classes, each of which was confirmed to behave as expected.

Table 5.1: Sample unit test cases and results

Test ID	Scenario	Expected result	Actual result
T-01	Create a sale for an available property	Property status changes to Sold; deposit recorded as an initial payment	Passed
T-02	Attempt to sell a property already marked Sold	Operation rejected with an explanatory error message	Passed
T-03	Delete an existing sale	Property status reverts to Available;	Passed

		associated payments removed	
T-04	Record a partial payment against a sale	Remaining balance decreases; payment status changes to Partial	Passed
T-05	Record payments summing to the full final price	Payment status changes to Paid; remaining balance equals zero	Passed
T-06	Delete a payment previously recorded against a sale	Remaining balance and payment status recalculated to reflect the deletion	Passed
T-07	Create a rental for a property already marked Rented	Operation rejected with an explanatory error message	Passed
T-08	Dashboard load with a contract expiring within thirty days	An expiring-contract notification is generated	Passed

5.2.2 User acceptance testing

User acceptance testing was carried out by exercising the complete system end to end using each of the six demonstration accounts described in the project's README file, one for each of the five roles, covering login, navigation, and the principal task associated with that role. Table 5.2 summarises the outcome of this testing.

Table 5.2: Sample user acceptance testing results

UAT ID	Role tested	Task performed	Outcome
--------	-------------	----------------	---------

UAT-01	Administrator	Log in, view dashboard, create a new staff user	Completed successfully
UAT-02	Project Manager	Create a project and assign a task to an employee	Completed successfully
UAT-03	Sales Agent	Register a new customer and record a property sale	Completed successfully
UAT-04	Accountant	Record a payment against an existing sale and review the payment report	Completed successfully
UAT-05	Customer	Log in and view own sale record and payment history	Completed successfully

The testing activities above confirmed that the core business rules identified in Chapter Four were correctly enforced across every implemented module, and that each of the five user roles could complete its intended tasks without encountering unexpected errors.

5.3 Screenshots and explanation

This section presents annotated screenshots of the completed system, captured directly from the running application using the Administrator account, in order to demonstrate that the functionality described in Chapter Four was successfully implemented.

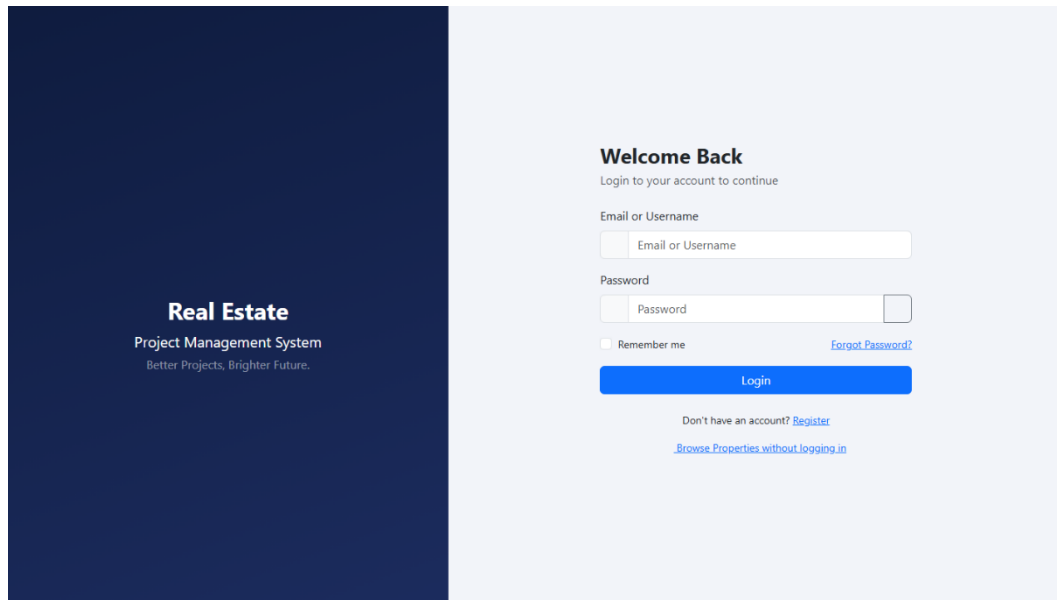


Figure 5.1: Login screen, showing the split-panel layout and the demonstration account credentials made available for evaluation purposes

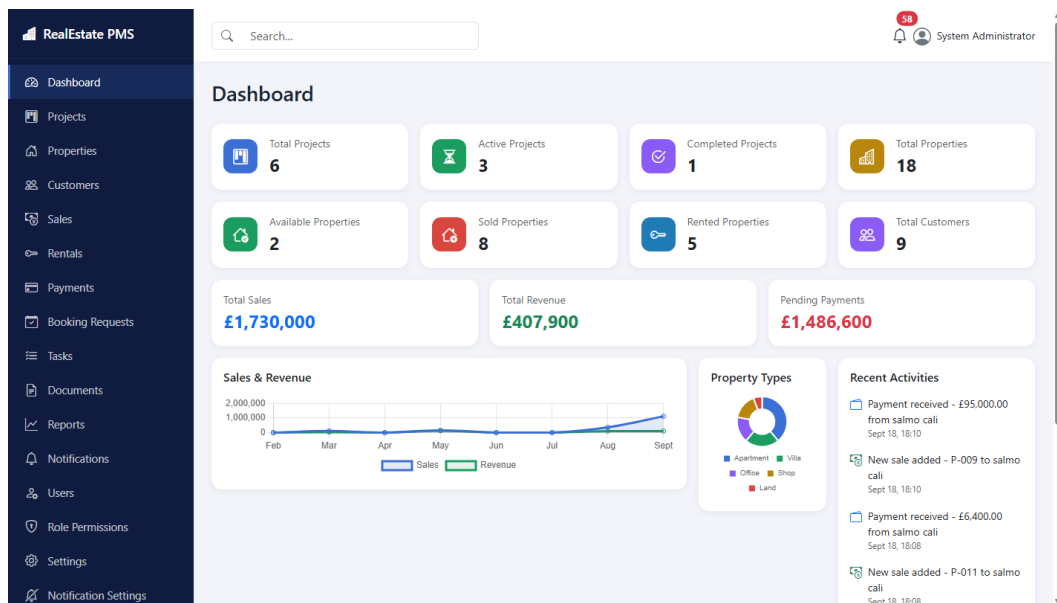


Figure 5.2: Administrator dashboard, summarising project, property, sales, and revenue statistics, together with a sales-and-revenue trend chart, a property-type distribution chart, and a recent-activity feed

RealEstate PMS

Search...

System Administrator

Properties + Add Property

Search properties... All Types All Statuses All Projects Filter

Image	Code	Project	Type	Price	Status	Actions
	P-015	Beach Front Villas	Villa	£480,000	UnderConstruction	
	P-014	Beach Front Villas	Villa	£500,000	Rented	
	P-013	City Center Complex	Villa	£380,000	Sold	
	P-012	City Center Complex	Apartment	£130,000	Rented	
	P-011	City Center Complex	Apartment	£140,000	Sold	
	P-017	Somali Plaza	Shop	£90,000	Rented	
	P-010	Somali Plaza	Office	£210,000	Sold	

Figure 5.3: Properties list, showing property code, associated project, type, price, and current status, with search and filter controls

RealEstate PMS

Search...

System Administrator

Add Property

Property Code:

Project: -- Select Project --

Property Type: Apartment

Building: Floor: Unit Number: Area (sqm): 0.00

Bedrooms: 0 Bathrooms: 0 Price: 0.00

Status: Available

Choose Images:

Description:

Figure 5.4: Add new property form, used to register a property under a selected project

RealEstate PMS

Search...

System Administrator

P-001 [Edit](#) [Back](#)

Sale Information

Customer: **salmo cali**

Final Price: **£115,000**

[View Sale](#)

Project: Green Valley Residences

Type: Apartment

Status: Sold

Building / Floor / Unit: Building A / 2 / A-2-1

Area: 120.00 sqm

Bedrooms / Bathrooms: 3 / 2

Price: £120,000

Created: Sept 14, 2026

Description: Modern apartment with sea view.

Figure 5.5: Property details view, showing full unit information alongside the linked sale record for a sold property

RealEstate PMS

Search...

System Administrator

Customers [+ Add Customer](#)

Search customers... All Countries [Filter](#)

Name	Phone	Email	Nationality	Actions
Khadija Omar	+252614445566	khadija.omar@example.com	Somali	View Edit Delete
Sagal Warsame	+252612223344	sagal.warsame@example.com	Somali	View Edit Delete
Amina Farah	+252617778899	amina.farah@example.com	Somali	View Edit Delete
Hassan Mohamed	+252615556677	hassan.mohamed@example.com	Somali	View Edit Delete
Yusuf Ali	+252613332222	yusuf.ali@example.com	Somali	View Edit Delete
Maryam Ahmed	+252614448888	maryam.ahmed@example.com	Somali	View Edit Delete
Mohamed Yusuf	+252611888888	mohamed.yusuf@example.com	Somali	View Edit Delete
Fatima Ali	+252611777777	fatima.ali@example.com	Somali	View Edit Delete

Previous 1 2 Next

Figure 5.6: Customers list, showing registered buyers and tenants

Customer	Property	Sales Agent	Sale Date	Final Price	Status	Actions
salmo cali	P-009	System Administrator	2026-09-18	£95,000	Paid	[View] [Edit] [Delete]
salmo cali	P-011	System Administrator	2026-09-18	£140,000	Partial	[View] [Edit] [Delete]
salmo cali	P-018	System Administrator	2026-09-18	£220,000	Partial	[View] [Edit] [Delete]
salmo cali	P-010	System Administrator	2026-09-18	£210,000	Pending	[View] [Edit] [Delete]
salmo cali	P-005	System Administrator	2026-09-18	£450,000	Pending	[View] [Edit] [Delete]
Yusuf Ali	P-013	Said Ibrahim	2026-08-14	£365,000	Partial	[View] [Edit] [Delete]
Maryam Ahmed	P-006	Mohamed Ali	2026-05-14	£135,000	Paid	[View] [Edit] [Delete]
salmo cali	P-001	Mohamed Ali	2026-03-14	£115,000	Partial	[View] [Edit] [Delete]

Figure 5.7: Sales management, listing recorded sales with customer, property, agent, sale date, final price, and payment status

Tenant	Property	Monthly Rent	Start	End	Payment	Contract	Actions
salmo cali	P-014	£2,500	2026-09-16	2027-09-16	Partial	Active	[View] [Edit] [Delete]
salmo cali	P-004	£150,000	2026-09-15	2027-09-15	Pending	Active	[View] [Edit] [Delete]
Amina Farah	P-012	£1,200	2026-08-14	2027-08-14	Partial	Active	[View] [Edit] [Delete]
Hassan Mohamed	P-017	£800	2026-07-14	2027-07-14	Partial	Active	[View] [Edit] [Delete]
Fatima Ali	P-003	£1,500	2026-06-14	2027-06-14	Partial	Active	[View] [Edit] [Delete]

Figure 5.8: Rentals management, listing active rental agreements

Customer	Type	Amount	Date	Method	Status	Actions
salmo cali	Sale #14	£95,000	2026-09-18	Cash	Completed	
salmo cali	Sale #13	£6,400	2026-09-18	Cash	Completed	
salmo cali	Sale #11	£8,000	2026-09-18	Cash	Completed	
salmo cali	Rental #5	£1,500	2026-09-16	BankTransfer	Completed	
Amina Farah	Rental #3	£2,400	2026-08-14	Cash	Completed	
Yusuf Ali	Sale #4	£100,000	2026-08-14	Cash	Completed	
Hassan Mohamed	Rental #2	£1,600	2026-07-14	Cash	Completed	
Fatima Ali	Rental #1	£3,000	2026-06-14	Cash	Completed	
Maryam Ahmed	Sale #2	£135,000	2026-05-14	Cash	Completed	
salmo cali	Sale #1	£20,000	2026-05-14	MobileMoney	Completed	

Figure 5.9: Payments list, showing individual payment transactions recorded against sales and rentals

#	Project Name	Location	Status	Progress	Actions
111	Guri Villa	Garowe	Planning	0%	
PRJ-002	Horizon Towers	Hargeisa	InProgress	25%	
PRJ-005	Beach Front Villas	Kismayo	OnHold	20%	
PRJ-004	City Center Complex	Mogadishu	InProgress	45%	
PRJ-003	Somali Plaza	Bosaso	Completed	100%	

Figure 5.10: Projects list, showing project code, location, status, and progress

RealEstate PMS

Search...

System Administrator

Project Tasks

+ Add Task

Search tasks... All Statuses All Projects Filter

Task	Project	Assigned To	Due Date	Priority	Status	Progress	Actions
Permit renewal	Beach Front Villas	Amina Yusuf	2026-09-24	Urgent	Pending		Edit Delete
Interior finishing	City Center Complex	Amina Yusuf	2026-11-14	Medium	InProgress		Edit Delete
Plumbing rough-in	City Center Complex	Amina Yusuf	2026-08-30	High	Delayed		Edit Delete
Final inspection	Somali Plaza	Amina Yusuf	2026-08-14	Medium	Completed		Edit Delete
Site survey	Horizon Towers	Amina Yusuf	2026-09-09	Urgent	Delayed		Edit Delete
Electrical installation	Green Valley Residences	Amina Yusuf	2026-10-14	Medium	InProgress		Edit Delete
Foundation works	Green Valley Residences	Amina Yusuf	2026-03-14	High	Completed		Edit Delete
Structural approval	Horizon Towers	Amina Yusuf	2026-10-04	High	Pending		Edit Delete

Figure 5.11: Project tasks, showing individual tasks assigned to employees within a project

RealEstate PMS

Search...

System Administrator

Documents

Upload Document

Search documents... All Types Filter

Title	Type	Related To	Uploaded By	Date	Actions
Payment Receipt - Payment #1	PaymentReceipt		System Administrator	2026-09-14	Download Delete
Project Document - Green Valley Residences	ProjectDocument	Project: Green Valley Residences	System Administrator	2026-09-14	Download Delete
Property Document - P-001	PropertyDocument	Property: P-001	System Administrator	2026-09-14	Download Delete
Rental Contract - Rental #1	RentalContract	Rental #1	System Administrator	2026-09-14	Download Delete
Sales Contract - Sale #1	SalesContract	Sale #1	System Administrator	2026-09-14	Download Delete

Figure 5.12: Documents list, showing uploaded contracts and receipts linked to their related records

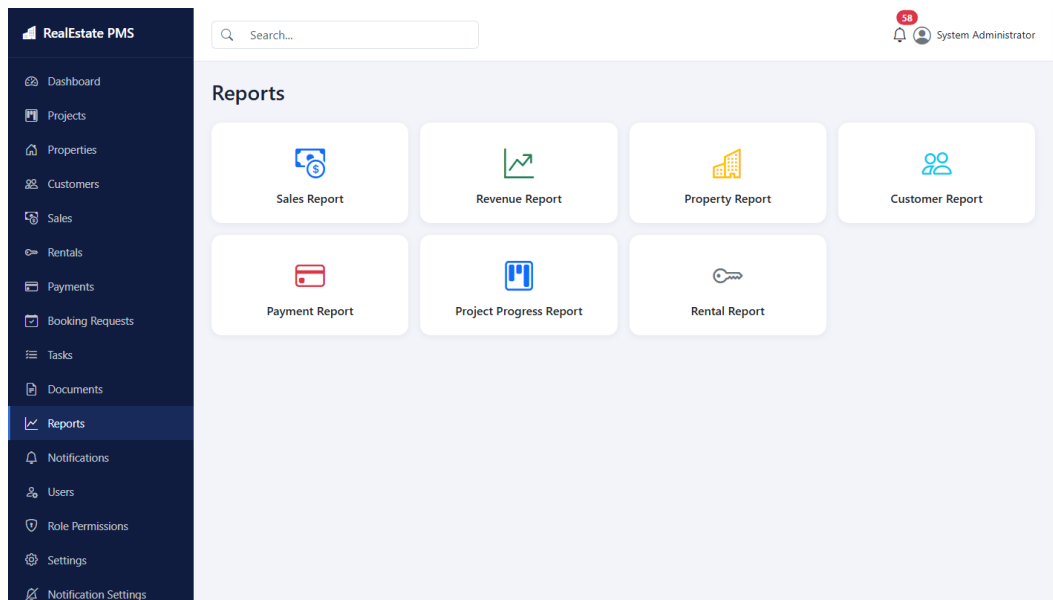


Figure 5.13: Reports hub, providing access to the sales, revenue, property, customer, payment, project-progress, and rental reports described in Section 4.6

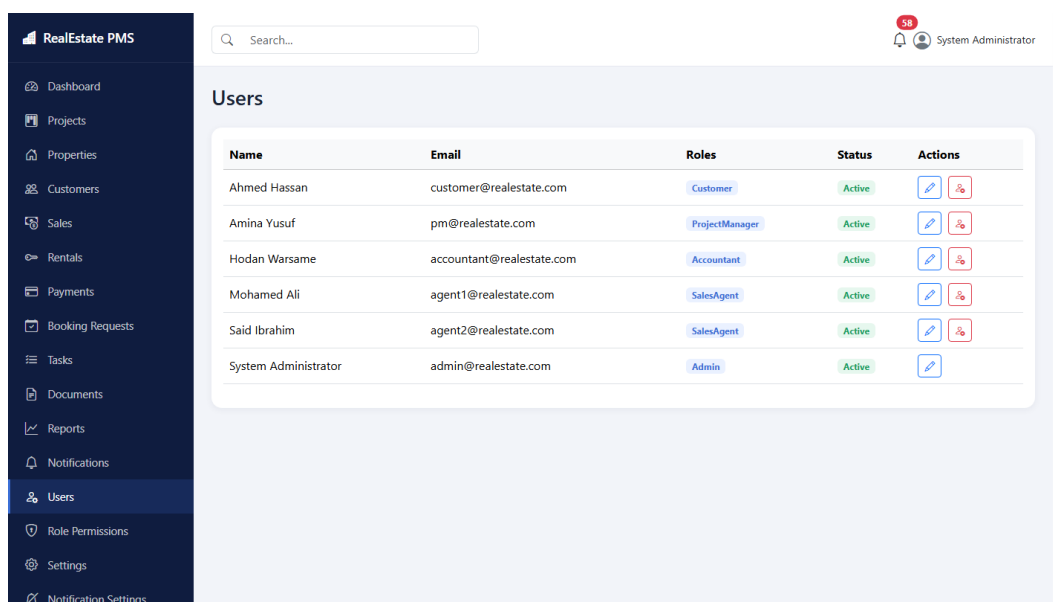


Figure 5.14: Users management, available to the Administrator role for creating and reviewing staff accounts

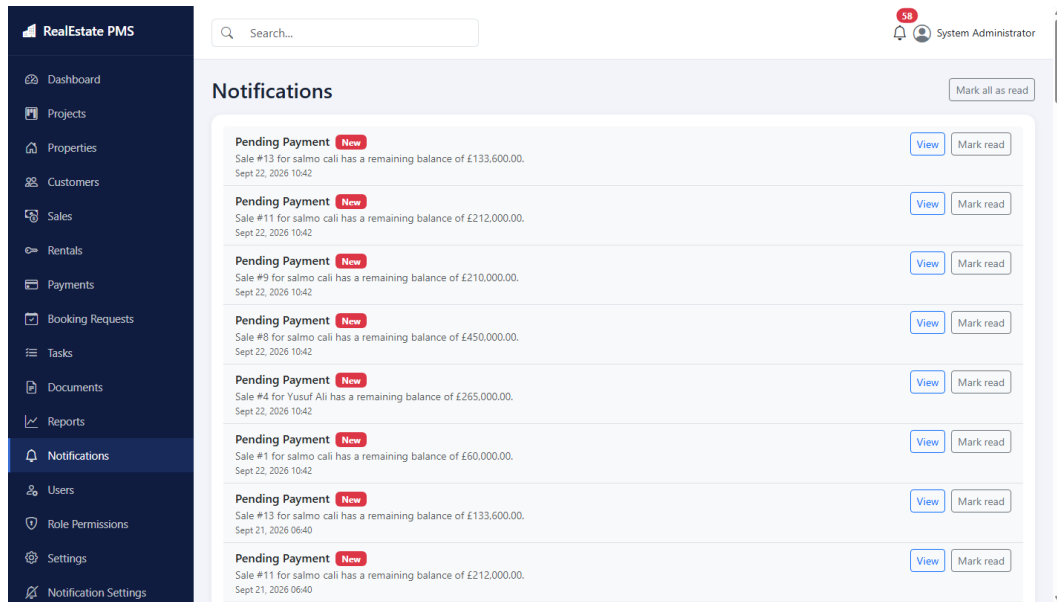


Figure 5.15: Notifications panel, displaying system-generated alerts for pending payments and other conditions described in Section 4.6

Taken together, the screenshots above confirm that every module identified in Section 4.6 was implemented and is fully operational, and that the role-based access control and automatic business rules described throughout Chapter Four function as designed in the completed application.

CHAPTER SIX: CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

This study set out to design, develop, and evaluate a web-based Real Estate Project Management System capable of replacing the manual and spreadsheet-based methods that many small real estate companies currently depend on. The completed system, built using ASP.NET Core 9, Entity Framework Core 9, and Microsoft SQL Server, integrates project tracking, property inventory, customer records, sales and rental transactions, payment processing, document storage, reporting, and system-generated notifications into a single, role-aware web application. Testing confirmed that the core business rules identified during requirements analysis, including automatic property status updates and automatic payment balance recalculation, function correctly, and that each of the five defined user roles can complete its intended tasks. The study therefore concludes that its general and specific objectives, stated in Section 1.3, were achieved.

6.2 Summary of findings

The requirements analysis confirmed that the central weaknesses of manual and spreadsheet-based real estate record keeping are outdated property status information, error-prone manual balance calculation, the absence of automatic warnings for overdue payments or expiring contracts, and scattered document storage, as discussed in Section 1.2. The system design and implementation described in Chapters Three and Four directly addressed each of these weaknesses: property status is now updated automatically whenever a sale or rental is created or removed, payment balances are recalculated automatically after every payment transaction, notifications are generated automatically for pending payments, expiring contracts, and delayed projects, and documents are stored centrally and linked to their related records. The testing results presented in Chapter Five confirmed that these mechanisms operate correctly across the demonstration data used for evaluation.

6.3 Recommendations and suggestions

Based on the findings summarised above, it is recommended that any real estate company considering adoption of a system of this kind first document its own specific business rules and approval workflows before configuring the software, since even a well-designed generic system cannot anticipate every local business practice. It is further recommended that staff receive brief, structured training on the system before its introduction, particularly regarding the payment recording workflow, since the accuracy of the automatically calculated balances depends entirely on every payment being entered promptly and completely. Finally, it is recommended that a company adopting this system

perform regular database backups, since, unlike a collection of paper files, the entire operational record of the company would reside in a single database.

6.4 Future enhancement

Several directions for future enhancement were identified but fall outside the scope defined in Section 1.4. A dedicated mobile application, or a responsive customer-facing portal, would allow customers to review their payment status without needing to contact staff directly. Integration with short message service and electronic mail providers would allow the existing in-application notifications described in Section 4.6 to also reach staff and customers outside the application itself. Integration with mobile money and banking application programming interfaces would allow payments to be recorded automatically rather than manually. Finally, extending the system to support multiple branches or multiple independent companies within a single deployment, using a tenant identifier added to the core entities described in Section 4.5, would broaden the system's applicability beyond the single-organisation scope adopted in this study.

REFERENCES

- Baum, A. (2020). *PropTech 2020: The future of real estate*. University of Oxford, Saïd Business School.
- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.
- Fowler, M. (2003). *Patterns of enterprise application architecture*. Addison-Wesley.
- Freeman, A. (2023). *Pro ASP.NET Core 8* (10th ed.). Apress.
- International Organization for Standardization. (2018). *ISO/IEC/IEEE 29148:2018 – Systems and software engineering – Life cycle processes – Requirements engineering*. ISO.
- Microsoft. (2025). *ASP.NET Core documentation*. <https://learn.microsoft.com/aspnet/core>
- Microsoft. (2025). *Entity Framework Core documentation*. <https://learn.microsoft.com/ef/core>
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill.
- Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.
- Stellman, A., & Greene, J. (2014). *Learning agile: Understanding Scrum, XP, Lean, and Kanban*. O'Reilly Media.

APPENDICES

Appendix A: Sample source code

The excerpt below, reproduced from Services/SalesService.cs, implements the business rule described in Section 4.6, under which creating a sale automatically updates the property's status and records any deposit as an initial payment.

```
public async Task<(bool Success, string? Error)> CreateSaleAsync(Sale
sale)

{

    var property = await _context.Properties.FindAsync(sale.PropertyId);

    if (property == null)

        return (false, "Property not found.");

    if (property.Status is PropertyStatus.Sold or PropertyStatus.Rented)

        return (false, "This property is not available for sale.");

    sale.FinalPrice = sale.TotalPrice - sale.Discount;

    sale.RemainingBalance = sale.FinalPrice - sale.Deposit;

    sale.PaymentStatus = CalculateStatus(sale.FinalPrice, sale.Deposit);

    property.Status = PropertyStatus.Sold;

    _context.Sales.Add(sale);

    await _context.SaveChangesAsync();
```



```

        if (sale.Deposit > 0)
        {
            _context.Payments.Add(new Payment
            {
                CustomerId = sale.CustomerId,

                SaleId = sale.Id,

                Amount = sale.Deposit,

                PaymentDate = sale.SaleDate,

                Method = PaymentMethod.Cash,

                Notes = "Initial deposit recorded automatically with the
sale."
            });

            await _context.SaveChangesAsync();
        }

        return (true, null);
    }

```

The excerpt below, reproduced from `Services/PaymentService.cs`, shows how recording or deleting a payment triggers automatic recalculation of the associated sale or rental balance, as described in Section 4.6.

```

public async Task<(bool Success, string? Error)>
CreatePaymentAsync(Payment payment)
{

```

```

        if (payment.SaleId == null && payment.RentalId == null)

            return (false, "A payment must be linked to a Sale or a
Rental.");

        _context.Payments.Add(payment);

        await _context.SaveChangesAsync();

        if (payment.SaleId.HasValue)

            await
_salesService.RecalculatePaymentStatusAsync(payment.SaleId.Value);

        if (payment.RentalId.HasValue)

            await
_rentalService.RecalculatePaymentStatusAsync(payment.RentalId.Value);

        return (true, null);

    }

```

Appendix B: Demonstration accounts used for testing

Role	E-mail	Password
Administrator	admin@realestate.com	Admin@12345
Project Manager	pm@realestate.com	Manager@12345

Sales Agent	agent1@realestate.com	Agent@12345
Accountant	accountant@realestate.com	Accountant@12345
Customer	customer@realestate.com	Customer@12345

Appendix C: Project folder structure

RealEstatePMS/

Controllers/ MVC controllers (one per module)

Models/

Entities/ EF Core entities

Enums/ Status / type enums

Data/ ApplicationDbContext + DbInitializer (seed data)

Repositories/ Generic repository (IRepository<T> /
Repository<T>)

Services/ Business logic (Sales, Rentals, Payments,
Notifications, Export, File storage, Dashboard)

ViewModels/ View-specific models

ViewComponents/ NotificationBell dropdown

Helpers/ PaginatedList<T>, StatusBadge

Views/ Razor views, organised by controller

wwwroot/ Static assets; uploaded files under
wwwroot/uploads

Migrations/ EF Core Code-First migrations

Appendix D: System screenshots

Additional system screenshots are included in Section 5.3 of this thesis.